the other two compilers against unrolling and distribution, and Clang is less stable when dealing with unroll-and-jam. The per-transformation results suggest that different compilers may have strengths and weaknesses in terms of stability when facing different source-level transformations.

To demonstrate the instability, we plot in log scale the ratio of the execution time of each loop's fastest mutation and its slowest one (Figure 4). The performance differences are taken from the 1,061 loops that are shared by all three compilers and then sorted for each compiler separately; thus, the data points at the same x-axis location do not necessarily represent the same loop. The plot clearly shows that ICC typically has the highest performance difference while Clang has the lowest when taking all transformation sequences into consideration.

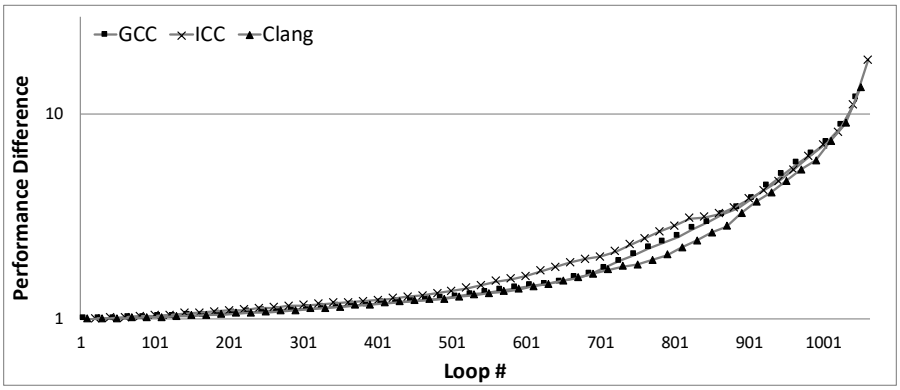


Fig. 4. Performance difference between each loop's best-to-worst mutations

Our results show that all three compilers have significant best-to-worst performance difference for the mutations of most loops, and the difference can be as high as 29x. Therefore, the compilers still have a long way to go before being stable.

4.7 Inter-compiler Stability

In Section 4.1, we mentioned that the performance of the code generated by the three compilers from the same loop may vary, meaning that apart from the intra-compiler stability that we discussed earlier, there is an additional inter-compiler stability to be concerned about. To quantify it, we propose an *inter-compiler stability score* S_{inter} to measure how close the effectiveness of the optimization processes of a set of compilers is, as given by,

$$S_{inter} = \frac{1}{L}\sum_{l=1}^{L}\frac{\sigma(t_C^{(l)})}{\mu(t_C^{(l)})} \quad C \in \{GCC, ICC, Clang\} \tag{4}$$

In the formula, $t_C^{(l)}$ is the execution time of baseline loop l compiled by compiler C. Like the S_{intra}, S_{inter} is an average of coefficient of variation. $S_{inter}^{(l)}$ of an individual loop l is computed by first taking the standard deviation of the execution time of the code generated by each compiler and then scaling it using their average execution time, so it measures how close the performances of a set of compilers' generated code are. S_{inter} of a set of loops L is the average of $S_{inter}^{(l)}$ where $l \in L$. The score is meaningful only when used in comparison, and a lower S_{inter} means closer performance. By comparing S_{inter} of different generations of the same set of compilers, one can track the progression of compiler stability in a bigger picture. However, in this paper we study only a single version of each compiler, so we use the score for another purpose: to investigate how